Kaveri Stays — API Assignment

Nine stages and a second reveal | Your database already enforces the rules | FastAPI, Swagger, Postman, JWT, pytest

You built the database. Now build the only way in. Designing the API surface is the assignment.

The Brief

Kaveri Stays has a real database now. It has stopped double-booking rooms, the guest list has one row per human being, and the owner can finally answer what last quarter's occupancy was.

What they do not have is a way for anyone except you to use it. Today the front desk at Coorg phones the owner, who opens `psql`. That is not a system.

The owner wants four things:

- The front desk working from a tablet at each property, taking bookings and checking guests in.
- Guests booking online without phoning anyone.
- Each property manager seeing their own property's numbers and nobody else's.
- Herself seeing everything.

Build the HTTP API that makes that possible. Postgres stays exactly as you left it. Write `def`, not `async def` — nothing in your twelve days of Python covered the event loop, and a blocking query inside an `async def` route stalls every other request to the API.

What you start with: your own `05_schema_final.sql` and `06_migration.sql`, plus the seed data from `07_seed.sql`. If your Stage 5 validator did not pass clean, fix that before you start. Everything here assumes your constraints actually work.

> Your database already refuses to double-book a room. It already caps guest count at the room type maximum, already rejects a rating of 9, already prevents two reviews on one stay. If you find yourself writing a Python function that SELECTs for overlapping bookings before it INSERTs, you have misunderstood this assignment as completely as you could have misunderstood the last one. The rules live in the schema. Your job is to carry a violation up to the client as the correct status code, and to leave no way around it.

Stage 1 — Read Your Own Constraints

Do not write any application code yet. Spend the first session in `psql`, reading what you already built.

```
SELECT conrelid::regclass AS table_name, conname, contype,
       pg_get_constraintdef(oid)
FROM   pg_constraint
WHERE  connamespace = 'public'::regnamespace
ORDER  BY 1, 2;
```

#	Task
1.1	Produce a full constraint inventory. One row per constraint: table, name, type, the business rule from the brief it serves, and the SQLSTATE it raises when violated. Find the SQLSTATEs by triggering each one deliberately, not by guessing.
1.2	For every constraint in that inventory, decide the HTTP status code a violation must produce. State how many distinct codes you ended up with. If the answer is one, start again.

#	Task
1.3	Three of your constraints will be violated by requests that are individually well-formed and only conflict with data already in the table. Name them. Explain why 400 is the wrong answer for all three.
1.4	Read the actual error text Postgres gives you for a failed exclusion constraint. Decide what part of it a client may see. Anything you pass through verbatim leaks your table and column names, and in this case leaks another guest's dates.
1.5	Rule 3 (guest count vs maximum occupancy) — check where you enforced it in 2.7. Depending on your answer then, it may or may not survive an INSERT that comes from the API. Verify by hand which it is.
1.6	The rule you admitted in 2.12 you could not fully constrain is now the API's problem. State exactly where it will live, and what happens when someone bypasses the API and uses `psql`.
1.7	Which of your 25 Stage 4 queries would be dangerous to expose directly as an endpoint, and why? Consider unbounded result sets, full-table scans on every request, and queries that span all three properties.
1.8	Name every column in your schema that a guest may never see. Columns, not tables.
1.9	Draw the booking state machine from rule 7. Five states. Mark every legal transition, and beside each one write which kind of person is allowed to perform it.
1.10	Written answer: name one thing about your schema you would change now that you know an HTTP API is going in front of it. If your answer is nothing, defend that.

> *1.5 is the question that separates the class. Rule 3 spans two tables, so where you put it in Stage 2 decides whether it holds here. Some of you will find it does not.*

Stage 2 — Identity and Tokens

Your `guests` table has rows with no password. Eleven years of them.

#	Task
2.1	Decide where credentials live: added to `guests`, or a separate accounts table that references it. Consider that staff are not guests, that most guests will never log in, and that a guest may one day be hired. Defend your choice in writing before you write DDL.
2.2	Model the four roles: guest, staff, manager, owner. A staff member and a manager each belong to exactly one property. The owner belongs to none.
2.3	Write the DDL for whatever you added. Same standard as last time — constraints, not comments. "Manager belongs to exactly one property, owner to none" is enforceable. Enforce it.
2.4	Hash passwords with bcrypt or argon2. State your cost factor and why you chose it. Time one login to show the cost is real.
2.5	`POST /auth/register` and `POST /auth/login`. Registration creates a guest account and nothing else, ever. Explain in one line why staff accounts must not be self-service.
2.6	List every claim in your access token, and for each one, why it is there. Then state what you deliberately kept out of it, and why anyone who can read a token can read all of it.
2.7	Refresh tokens: longer expiry, stored server-side, rotated on use. `POST /auth/refresh` and `POST /auth/logout`.
2.8	A manager is fired at 10:00. Their access token expires at 10:15. Decide what happens in those fifteen minutes. Implement your decision, and name what it costs you.

#	Task
2.9	Property scope: inside the token, or looked up per request? Pick one, then work out what happens when a manager is transferred from Ooty to Coorg mid-shift under each option.
2.10	No secret in the repository. `.env`, `.env.example`, `.gitignore`. If `SECRET_KEY` is missing at startup the application must refuse to start, loudly. Show that it does.
2.11	Make Swagger usable: protected endpoints show a padlock, and the Authorize button accepts a real token. Your juniors' juniors will thank you.
2.12	Written answer: HS256 or RS256 for this system. Pick one, defend it, and name the change to Kaveri Stays that would make you switch.

Stage 3 — Design the Surface

This is the heart of it, exactly as Stage 2 was last time. You are writing a specification, not an application. No route handlers yet.

#	Task
3.1	Write the complete OpenAPI spec by hand as `03_openapi_original.yaml`. Every path, every method, every request and response model, every status code each endpoint can return, and which of the four roles may call it.
3.2	Availability is the endpoint the booking desk hits all day. Design its path and query parameters. It is your 4.1 query over HTTP, so decide now how a caller asks for one property, one date range, and optionally one room type.
3.3	Design the booking lifecycle endpoints. One `PATCH /bookings/{id}` carrying a new status, or separate `/check-in`, `/check-out`, `/cancel` actions. Pick one and argue it. Your state machine from 1.9 is the thing being modelled.
3.4	Design payments. One booking, several payments, rule 8. Decide how a caller signals that a retried request is the same payment and not a second one.
3.5	Design the reporting endpoints for occupancy, ADR and RevPAR. Decide whether they hang off `/properties/{id}/...` or a separate `/reports` tree, and what that choice does to your authorization rules.
3.6	Every list endpoint needs pagination. Choose offset or keyset, apply it uniformly, and state what happens with offset pagination on page 400.
3.7	Decide once, for the whole API: does an empty result set return 200 with an empty list, or 404? Write the rule down. Apply it everywhere without exception.
3.8	Decide once: 401 or 403. Write the rule that tells them apart, and make sure a missing token and an insufficient role do not produce the same answer.
3.9	Every date crossing your API boundary uses one format. State it. Then explain why check-in is a date and not a timestamp, and what you would have to change if Kaveri Stays opened a property in another timezone.
3.10	Design one error envelope for the entire API. Every failure, from a validation error to a constraint violation to an expired token, comes back in this shape. Write the schema.
3.11	Fill in the full authorization matrix: every endpoint down the side, four roles across the top, every cell either allowed or denied. No blanks.
3.12	Written answer: name one endpoint you decided not to build, and why. Deleting a booking is a candidate. So is anything that lets a client set a price.

> *3.11 is tedious and it is where the marks are. A matrix with a hole in it is how every real authorization bug gets shipped.*

Before you go any further — Mail your spec

Commit `03_openapi_original.yaml` and your authorization matrix, then mail them. You are about to be shown one possible answer. If you can quietly edit your work first, the next section is worth nothing to you and nothing to me.

Interlude — The Second Reveal

Open `openapi_reveal.yaml` only now, with Stage 3 committed and timestamped.

It contains the paths, payloads and status codes that Stages 4 to 8 and the automated test suite assume. It is one reasonable design. It is not the only one, and unlike the schema reveal, several of its choices are genuinely arguable.

#	Task
R1	Confirm your Stage 3 spec and matrix are committed and timestamped. Do not continue until they are.
R2	For every path in the reveal, record whether you have the same thing, the same thing at a different path, or nothing equivalent.
R3	The reverse: list every path in your spec the reveal does not have. Explain what it is for.
R4	Compare status codes endpoint by endpoint. Where you chose differently, argue your case. Where you returned 400 and the reveal returned 409, work out which of you is wrong.
R5	Where the reveal has something you missed, name the business rule or the attack it exists to serve, and what would have broken without it.
R6	Rename your paths and fields to match the reveal so the test suite and the remaining stages can run. Keep `03_openapi_original.yaml` untouched and submit both.

How to read your differences

Category	What it means
Arguable	Whether status transitions are one PATCH or four action endpoints. Whether credentials extend `guests` or sit in their own table. Offset versus keyset pagination. Whether reporting lives under `/reports` or under each property. A well-argued alternative scores full marks here.
Not arguable	401 versus 403. A guest reading another guest's booking. A nightly rate supplied by the client. A role read from a request body. Two concurrent bookings for the same room both succeeding.
Telling	If every endpoint in your spec returns only 200 and 400, you designed the happy path and stopped. Designing an API is mostly designing its failures.

Stage 4 — Build the Read API

#	Task
4.1	The availability endpoint. Your 4.1 query, unchanged, behind HTTP. Boundary dates must behave: a room whose previous guest checks out on the 15th is available from the 15th.

#	Task
4.2	`GET /bookings` with filtering by property, status, date range and guest, and sorting. Sortable fields come from a whitelist. Explain what happens if they do not.
4.3	`GET /bookings/{id}` where the booking exists but belongs to a different guest. 404 or 403? Decide, and defend it against the opposite argument.
4.4	Occupancy, ADR and RevPAR endpoints. The arithmetic stays in SQL. If you find yourself summing revenue in a Python loop, you have thrown away Stage 4 of the last assignment.
4.5	`GET /me`. Returns the caller's own record, shaped by their role.
4.6	No endpoint returns a raw database row. Every response goes through a response model. Prove with a test that no path anywhere in the API can emit a password hash.
4.7	Rooms that have never been booked still exist. Make sure your availability and inventory endpoints show them. Several of your Stage 4 queries depended on this.
4.8	Written answer: which of your read endpoints lets a guest discover the existence of other guests? Room occupancy is a candidate. Show the fix.

Stage 5 — Writes and Transactions

#	Task
5.1	`POST /bookings` as one transaction: check the room type allows the guest count, insert the booking, record the deposit payment. This is 5.22 from the last assignment, wearing a URL. All of it commits or none of it does.
5.2	Catch the exclusion constraint violation and return 409 with a body that tells the caller the room is taken without telling them who has it or on what dates.
5.3	Map every SQLSTATE from 1.1 to a status code in exactly one place in your codebase. One handler. If you have `except IntegrityError` in more than one file, consolidate.
5.4	Implement the state machine from 1.9. Illegal transitions are refused. Confirmed to checked-out without passing through checked-in is refused.
5.5	Enforce who may perform each transition. A guest may cancel their own booking. Only staff may check anyone in.
5.6	`POST /bookings/{id}/payments`. Instalments, per rule 8. A retried request with the same idempotency key records one payment, not two. Explain what happens without one when the front desk tablet loses signal mid-request.
5.7	Decide what your API does when payments would exceed the booking total. Then say where that rule belongs — API, constraint, or neither — and why.
5.8	`POST /bookings/{id}/review`. Only after checkout, only once. Both are already constraints. Return the right code for each, and explain why they are not the same code.
5.9	The nightly rate comes from `rate_plans`, looked up server-side. Never from the request body. Write one sentence describing what a guest could do if it came from the body.
5.10	A booking spanning Christmas crosses two rate periods. Decide what your API charges, document the decision, and make sure the total it stores agrees with the payments it accepts.
5.11	No endpoint hard-deletes a booking. Cancellation is the mechanism. Tie this back to the ON DELETE choices you defended in 2.11.

#	Task
5.12	Written answer: show your transaction boundary. Name the line where commit happens. Then force the payment insert to fail after the booking insert has succeeded, and show the database afterwards.

Stage 6 — Drive It From Outside

Everything up to here you could have tested with `curl` and your own memory of what you built. From now on, two other people have to be able to use this API: the front desk, and whoever maintains it after you leave. That is what Swagger and Postman are for.

#	Task
6.1	FastAPI generates a spec at `/openapi.json`. Diff it against the `03_openapi_original.yaml` you wrote by hand. List every difference. Each one is either a spec you got wrong or an implementation that drifted. Say which, for each.
6.2	Every status code an endpoint can actually return must be declared in its decorator. An endpoint that returns 409 in production and shows only 200 in `/docs` is lying to the front desk. Audit all of them against your Stage 1 mapping.
6.3	`response_model` on every route. Then open `/docs` and confirm the documented response for `GET /bookings/{id}` contains no field from your 1.8 list.
6.4	Tags, summaries and descriptions so `/docs` reads as something a non-programmer can follow. The owner is going to open this page and try to book a room from it.
6.5	Build a Postman collection covering every endpoint in the API. One folder per resource.
6.6	Four Postman environments: guest, staff, manager, owner. Each holds its own credentials and its own `token` variable. Switching role becomes switching environment.
6.7	A post-response script on the login request that writes the access token straight into the environment variable. After this, no request anywhere in your collection contains a pasted token.
6.8	Set authorization once at collection level so every request inherits it. Then explain what happens to a colleague who runs your collection tomorrow if a single request has its own hard-coded token instead.
6.9	A `pm.test` on every request asserting its status code. Run the whole collection through the Collection Runner and report the pass and fail counts.
6.10	A folder called `take a booking` that runs end to end in order: log in, check availability, create the booking, pay the deposit, check in, check out, leave the review. Every id comes from the previous response, captured in a script. No hard-coded ids anywhere in it.
6.11	Export the collection and all four environments for submission. The exports must contain no real password or secret. Say what you did instead, and how a new joiner gets a working environment on their first day.
6.12	Written answer: three things now describe this API — `05_openapi_final.yaml`, the spec FastAPI generates, and your Postman collection. Name which one is authoritative, and what you would put in place to stop the other two drifting from it.

> *6.7 is the whole reason you were made to copy a token by hand in Swagger until it became unbearable. If you are still pasting tokens at the end of this stage, you have not finished it.*

Stage 7 — Authorization

Your collection from Stage 6, with its four environments, is now the tool you test this stage with. Every claim below should be a request you can run.

#	Task
7.1	Implement the matrix from 3.11 in full.
7.2	Property scoping lives in a dependency, not in an `if` at the top of each route. Explain what goes wrong when it lives in the route.
7.3	The Ooty manager cannot read Coorg's revenue. Not filtered out of the response — refused.
7.4	Object-level ownership: a guest sees their own bookings, payments and reviews, and no others.
7.5	The owner's cross-property reports are reachable by the owner and by nobody else, including managers.
7.6	Your model from 2.1 either does or does not allow one person to be both a guest and a staff member. Say which, say whether that was deliberate, and say what a booking made by that person means.
7.7	Run one request from all four environments in turn. The four responses are your matrix, proved. Do this for at least six endpoints and paste the grid.
7.8	Written answer: for each read endpoint, say whether the authorization check happens before the query or inside it as a WHERE clause. Then explain the endpoint where inside is the only correct answer.

Stage 8 — Try to Break It

Same rules as last time. Every attack below must be refused. Build them as a folder called `attacks` in your Stage 6 collection, each with a `pm.test` asserting the refusal, so the whole suite runs in one click. Paste the exact response, status code and body, and name what stopped it. If any of them succeeds, you have a hole.

#	Attack
8.1	Guest A requests guest B's booking by id.
8.2	Register an account with `"role": "owner"` in the request body.
8.3	Present a token with its algorithm set to `none`.
8.4	Present a token signed with a different secret.
8.5	Present an expired access token.
8.6	Reuse a refresh token that has already been rotated.
8.7	Use the Ooty manager's valid token against Coorg's revenue endpoint.
8.8	Create a booking with `nightly_rate` supplied in the request body.
8.9	Post a review on a booking that is still checked in.
8.10	Two concurrent `POST /bookings` for the same room and overlapping dates. Genuinely concurrent, not one after the other. Report both responses and both status codes.
8.11	SQL injection through your sort parameter, then through your filter parameter.

#	Attack
8.12	Delete your Pydantic validation on guest count, then send four guests to a room whose type allows two. It must still fail. Name what caught it.
8.13	Two hundred login attempts against one email address.
8.14	Work out which email addresses are registered by comparing login responses.

And these must be ALLOWED. If your API refuses them, it is too strict.

#	Must succeed
8.15	A guest checks out on the 5th and another checks into the same room on the 5th, both through the API.
8.16	A cancelled booking exists for 1–5 March; a different guest books that room for 2–6 March.
8.17	One booking receives three separate part-payments.
8.18	One guest holds bookings at all three properties simultaneously.

#	Task
8.19	One of the attacks above cannot be run from Postman at all, no matter how you arrange the collection. Identify it, explain why the Collection Runner cannot express it, and write it in Python instead.
8.20	Written answer: sort 8.1 through 8.14 into two lists — caught by the database, and caught only by the API. Then explain why everything in the second list is a standing risk, and what you would need to move any of it into the first.

> *8.10 is this assignment's concurrency test, and it is the reason the last one had one too. Run it with two real parallel requests. If both come back 201, you have reproduced the original spreadsheet bug through a brand new API.*

Stage 9 — Make It Hold Up

#	Task
9.1	Find an N+1 query in your own code. Report the query count before and after fixing it, measured, not estimated.
9.2	State your connection pool size and demonstrate what the API does when the pool is exhausted.
9.3	Rate limit `POST /auth/login`. State the limit and what a blocked caller receives. Then re-run attack 8.13 from Postman and show it now fails.
9.4	Run `EXPLAIN ANALYZE` on the query behind your availability endpoint and confirm it still uses the index you added in 6.3 of the database assignment. If it does not, work out what the API changed.
9.5	A pytest suite covering every item in Stages 5 and 8. Report coverage. Tests that only exercise the happy path do not count.
9.6	Written answer: you now have Postman tests and pytest tests over the same API. Draw the line between them. Say which belongs in a pipeline that runs on every commit, and why the other one cannot go there.
9.7	Written answer: your API is now the way in, except it is not, because staff still have `psql`. Name the smallest change that makes the API the only path, and name what Kaveri Stays loses by making it.

What to Hand In

File	Contents
`01_constraints.md`	Your Stage 1 inventory, the SQLSTATE-to-status mapping, and your four written answers.
`02_auth_design.md`	Your identity model, defended, plus your answers to 2.6, 2.8, 2.9 and 2.12.
`02_auth_schema.sql`	The DDL for accounts, roles and property assignment.
`03_openapi_original.yaml`	Your spec exactly as committed before you opened the reveal. Do not tidy it.
`03_authorization_matrix.md`	The full matrix from 3.11, as committed.
`04_reconciliation.md`	The Interlude: what matched, what differed, and your arguments where you still prefer your own design.
`05_openapi_final.yaml`	The renamed spec the rest of your work runs against.
`app/`	The application. Routers, models, dependencies, one error handler, no secrets.
`06_spec_drift.md`	Your 6.1 diff between the hand-written spec and the generated one, and your answer to 6.12.
`06_postman_collection.json`	The exported collection, including the `attacks` folder and the `take a booking` flow.
`06_postman_environments/`	All four environments, exported, with no real secrets in them.
`07_authorization_matrix.md`	The matrix as implemented, plus the four-environment grid from 7.7.
`08_break_it.md`	Every Stage 8 attempt, the exact response, and what stopped it. Include your Collection Runner output.
`09_performance.md`	Stage 9: query counts, EXPLAIN output, and your three written arguments.
`README.md`	How to run it, in order, from an empty database. Your design decisions, and anywhere you deliberately did something the reveal does not do.

Marking

Stage	Marks	What is being judged
Stage 1	10	Did they read their own schema, or start writing routes?
Stage 2	10	Is identity modelled, or bolted on?
Stage 3	20	The heart of it. Did they design failures as carefully as successes?
Interlude	5	Honesty and the quality of the argument, not the number of matches.
Stage 4	5	Reads that are correct at the boundaries and leak nothing.
Stage 5	15	Transaction boundaries, and constraint violations translated rather than pre-empted.
Stage 6	10	Could a stranger run this API tomorrow from the collection and the docs alone?
Stage 7	10	Authorization that is structural rather than sprinkled.
Stage 8	10	Attack suite score, plus whether they understood 8.10, 8.19 and 8.20.

Stage	Marks	What is being judged
Stage 9	5	Evidence they can measure rather than guess.

Reading the result	What it looks like
Strong signal	One exception handler mapping SQLSTATEs to status codes. Authorization as a dependency, with property scope pushed into the WHERE clause. Reached for an idempotency key on payments unprompted. Noticed the client-supplied price before 7.8 told them. Distinguished 401 from 403 without being reminded. Argued a spec difference against the reveal and was right. A collection a stranger could clone and run start to finish without asking a question.
Weak signal	Every failure is a 400. A `SELECT` for overlapping bookings before the `INSERT`. Authorization by `if` statement at the top of each route. Role taken from the request body. Tokens without expiry. Availability endpoint returning rooms that are occupied on the boundary date. Tokens pasted by hand into every Postman request. `/docs` promising 200 where the code returns 409.
Fail	`SECRET_KEY` committed to the repository. `verify_signature=False` anywhere. Both concurrent bookings in 7.10 succeed and it is written up as a race condition to fix later. Or the exclusion constraint was quietly dropped so the API could do the checking. Or a real password shipped inside an exported Postman environment.

The last assignment asked whether your schema enforces the business or merely stores it. This one asks a narrower question: having built a database that cannot be talked into a double booking, can you put a door on it that cannot either.